Ejercicio práctico UT2 Despliegue de un Proxy Inverso Nginx con Terminación SSL sobre AWS EC2 mediante Docker Compose

Contenido

Ejercicio práctico UT2 Despliegue de un Proxy Inverso Nginx con Terminación SSL sobre AWS EC2 mediante Docker Compose 0
Introducción 1
¿Qué es Nginx? 1
¿Qué es un Proxy Inverso? 1
Objetivo 3
Escenario 3
Tareas a realizar 3
1. Crear la instancia EC2 3
2. Automatizar la instalación 4
3. Crear la aplicación Flask 4
4. Crear el fichero Docker Compose 4
Contenedor Nginx 4
Contenedor Flask 4
Red Docker 5
5. Configurar la terminación SSL 5
6. Configurar la redirección HTTP → HTTPS 5
7. Comprobar el funcionamiento 5
Entrega 5
Rúbrica 6
Criterios de evaluación 6
1. HTTPS operativo (2 puntos) 6
2. Redirección HTTP → HTTPS (1 punto) 6
3. TLS 1.0 deshabilitado (1 punto) 7
4. TLS 1.2 o superior habilitado (1 punto) 7
5. Aplicación Flask accesible mediante el proxy (1 punto) 7
6. Video explicativo 2 puntos 7
Bonus (+1 punto) 7
Publicación de IPs para la corrección 8

Introducción

En arquitecturas web modernas es habitual separar la aplicación en varias capas para mejorar la seguridad, escalabilidad y facilidad de mantenimiento.

¿Qué es Nginx?

Nginx es un servidor web muy utilizado en entornos profesionales. Además de servir páginas web, puede actuar como:

- Proxy inverso.
- Balanceador de carga.
- Terminador SSL/TLS.
- Servidor de contenido estático.

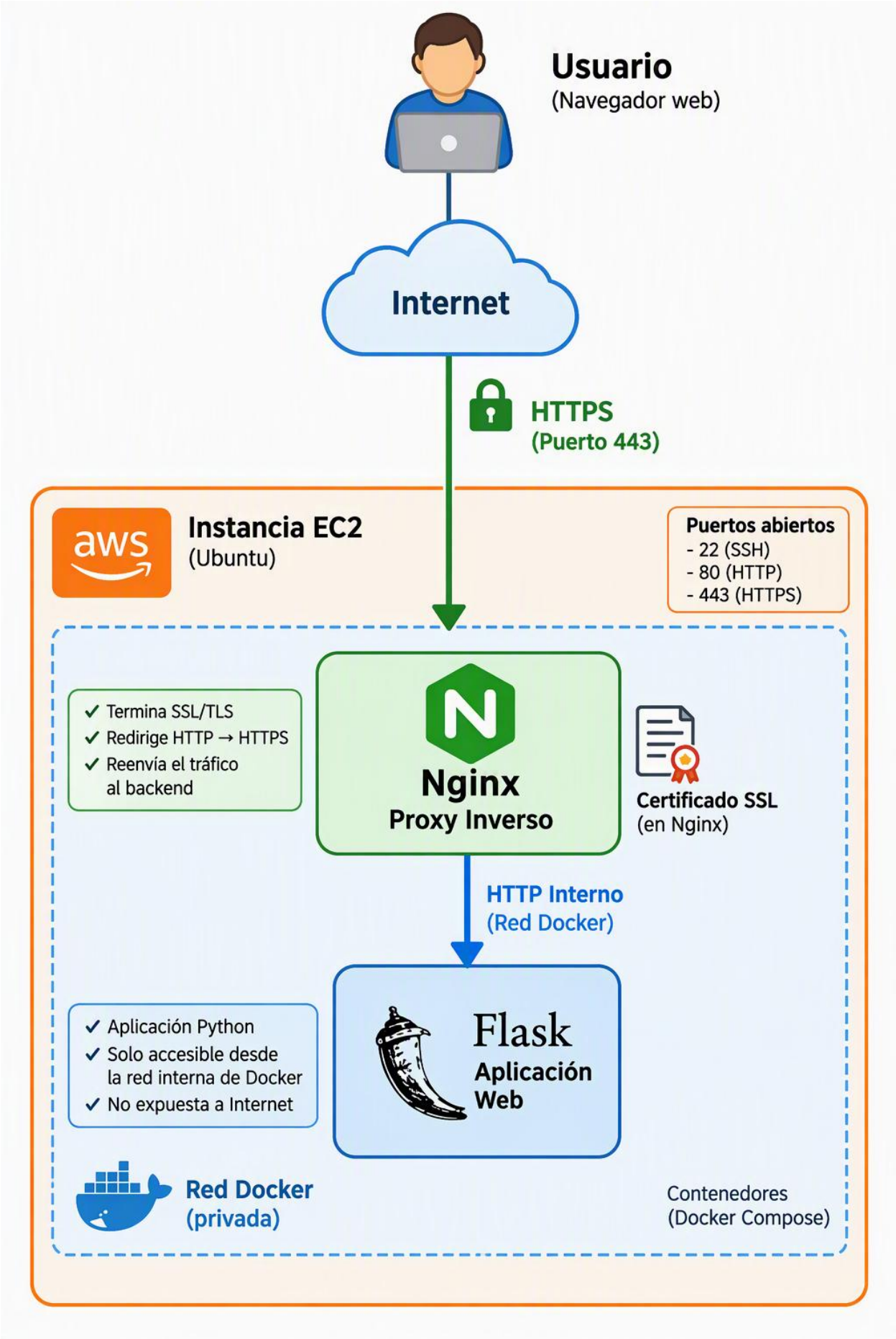
Actualmente es una de las soluciones más utilizadas para publicar aplicaciones desarrolladas con Flask, Node.js, Django y otras tecnologías.

¿Qué es un Proxy Inverso?

Un **proxy inverso** es un servidor que recibe las peticiones de los usuarios y las reenvía a uno o varios servidores internos.

La ventaja principal es que los usuarios nunca acceden directamente a la aplicación.

Arquitectura del despliegue:


Usuario
(Navegador web)
Internet
HTTPS
(Puerto 443)
aws
Instancia EC2
(Ubuntu)
Puertos abiertos
- 22 (SSH)
- 80 (HTTP)
- 443 (HTTPS)
✓ Termina SSL/TLS
✓ Redirige HTTP → HTTPS
✓ Reenvía el tráfico al backend
Nginx
Proxy Inverso
Certificado SSL
(en Nginx)
HTTP Interno
(Red Docker)
✓ Aplicación Python
✓ Solo accesible desde la red interna de Docker
✓ No expuesta a Internet
Flask
Aplicación Web
Red Docker
(privada)
Contenedores
(Docker Compose)

De esta forma:

- Nginx gestiona HTTPS.
- Nginx recibe todas las peticiones externas.
- Flask permanece aislado en una red interna.
- La superficie de ataque es menor.
- Se simplifica la gestión de certificados SSL.

Objetivo

Desplegar una arquitectura web formada por:

- Una instancia AWS EC2.
- Un Proxy Inverso Nginx.
- Una aplicación Flask.
- Una red privada Docker.
- HTTPS mediante terminación SSL en Nginx.

Todo el proceso deberá quedar automatizado mediante **User Data**, de forma que la instancia quede operativa automáticamente al arrancar.

Escenario

Una empresa desea publicar una aplicación web corporativa en Internet.

Como administrador de sistemas debes desplegar una solución basada en contenedores que garantice:

- Separación entre frontend y backend.
- Uso de HTTPS.
- Protección de la aplicación mediante un proxy inverso.
- Automatización del despliegue.

Tareas a realizar

1. Crear la instancia EC2

Crear una instancia Ubuntu en AWS EC2.

El Grupo de Seguridad deberá permitir:

- TCP 22 (SSH)
- TCP 80 (HTTP)
- TCP 443 (HTTPS)

---

2. Automatizar la instalación

Diseñar un script: user-data.sh que se ejecute en el primer arranque de la instancia.

El script deberá instalar automáticamente:

- Docker Engine.
- Docker Compose.

No se permite realizar la instalación manual una vez creada la máquina.

---

3. Crear la aplicación Flask

Desarrollar una aplicación Flask sencilla.

La página principal deberá mostrar obligatoriamente:

- Nombre y apellidos del alumno.
- Centro formativo-ciclo-grupo.
- Fecha de despliegue.

---

4. Crear el fichero Docker Compose

Crear el fichero: YAML ***docker-compose.yml*** que despliegue:

Contenedor Nginx

- Escucha en el puerto 80.
- Escucha en el puerto 443.
- Gestiona las conexiones HTTPS.
- Actúa como Proxy Inverso.

Contenedor Flask

- Ejecuta la aplicación web.
- No deberá ser accesible desde Internet.

Red Docker

Ambos contenedores deberán comunicarse mediante una red privada Docker.

---

5. Configurar la terminación SSL

El contenedor Nginx deberá:

- Recibir conexiones HTTPS.
- Gestionar el certificado SSL.
- Descifrar las conexiones TLS.
- Reenviar el tráfico al contenedor Flask.

---

6. Configurar la redirección HTTP → HTTPS

Toda petición recibida por HTTP deberá redirigirse automáticamente a HTTPS.

http://IP_PUBLICA  https://IP_PUBLICA

---

7. Comprobar el funcionamiento

La aplicación deberá estar accesible mediante navegador utilizando:

https://IP_PUBLICA o https://DOMINIO si se utiliza un nombre de dominio.

Entrega

Cada alumno deberá entrega en el aula virtual:

1. Enlace al repositorio de GitHub privado con

    - user-data.sh
    - docker-compose.yml

- configuración de Nginx
- código fuente de Flask

2. Enlace a video corto en mediateca (menos de un minuto) en que se vea la configuración y la página web en el navegador
3. PDF con pantallazos de la configuración y el navegador con la IP Pública. El pdf debe contener los pantallazos de las comprobaciones de cada uno de los criterios de evaluación.

Rúbrica

Apartado	Puntos
PDF con la configuración y las 5 comprobaciones 1- HTTPS operativo 2- Redirección HTTP → HTTPS 3- TLS 1.0 deshabilitado 4- TLS 1.2 o superior 5- Página web con flask	5
Git con readme y archivos solicitados	2
Video explicativo	2
Bonus (pantallazo en pdf de configuración y validación)	1

Criterios de evaluación

1. HTTPS operativo (2 puntos)

Comprobación:

Shell
curl -k -I https://IP

Resultado esperado: HTTP/1.1 200 OK

---

2. Redirección HTTP → HTTPS (1 punto)

Comprobación:

Shell
curl -I http://IP

Resultado esperado: 301 Moved Permanently o 302 Found

---

3. TLS 1.0 deshabilitado (1 punto)

Comprobación:

Shell
openssl s_client -connect IP:443 -tls1

La conexión debe fallar.

---

4. TLS 1.2 o superior habilitado (1 punto)

Comprobación:

Shell
openssl s_client -connect IP:443 -tls1_2

La conexión debe completarse correctamente.

---

5. Aplicación Flask accesible mediante el proxy (1 punto)

La página web debe mostrar la información identificativa del alumno.

---

6. Video explicativo 2 puntos

Bonus (+1 punto)

Configurar un dominio propio y obtener un certificado válido mediante Let's Encrypt.

Evidencia:

Shell
curl -Iv https://midominio.com

sin advertencias de certificado ni errores de validación.

Publicación de IPs para la corrección

Se habilitará un formulario para introducir las URLs de GiT y del video de mediateca.

Se habilitará un Excel para la validación de las IPs.

Alumno	IP Pública
Nombre Apellidos	XX.XX.XX.XX

Todos los alumnos deberán registrar su dirección IP cuando lo solicite el profesor.

La evaluación se realizará directamente sobre los servicios desplegados.

La instancia deberá permanecer activa durante el periodo de corrección.